CSE 112 – Lab #11 – STL Stack Container Operations –Vector Implementation

Reference Chapter 19 page 1241 and 42

A common detection mechanism in data communication and processing uses an eighth bit added to ASCII characters to indicate the parity. This *parity bit* is an extra bit that makes the total number of 1's either even or odd depending upon the parity (even or odd). Even parity is more common, and a 1 or a 0 is added as the (left-most) eighth bit to make the number of 1's even.

	With even parity
ASCII A = 1000001	01000001
ASCII T = 1010100	11010100

Even Parity

Write a program that implements an **STL Stack container** (vector, list, or deque) that reads a string of binary numbers (provided below) and pushes eight digits on the stack at a time while counting the number of 1's. If the number of 1's is odd, pop those eight values and output "negative acknowledge # x". Where "x" is the count of negative acknowledges. If the number of 1's is even, process the next eight digits. When the end of the file is reached, display the size of the stack and the values on the stack seven digits per line (omit the parity bit).

Note: the vector implementation is the most common

```
stack<int, vector<int>> myStack;      // STL vector stack per page 1193
stack<int, list<int>> myStack;        // STL list stack per page 1193
stack<int> myStack;                   // Default - STL deque stack per page 1193
```

Stack member functions on page 1193.

The solution can be implemented in about 65 lines of code. Do not go down a rabbit hole and make this hard. Engineer the solution.

Grading is based on meeting all of the lab requirements, adherence to programming standards, operation and accurate output, and programming style including white space and indentation. The grade penalty is 5% for each standards violation or missing requirement.

Binary string:

0100001000010111100011111100001110011011010100110010011101111001011100001